

SRW3 Phase 1I — Protocol-Authoritative Commitment Semantics

Branch `phase1i`, created additively from `phase1h-r1-fix` @

`bf78b8c034f7bcab83cda2862be288d8524f824c` (tag `phase1h-r1-complete`).

Frozen branches (`phase1g`, `phase1g-complete`, `phase1h`, `phase1h-r1-fix`)

are untouched: no rewrite, no amend, no force-push; verified before and after the work by `git diff` being empty on all frozen paths and by branch SHAs being unchanged on the remote.

1. Research question

Phase 1H-R1 established experimentally that a real Ethereum execution client (Geth 1.17.7, unmodified) can execute a payload, that the Engine API boundary can be observed through client-derived evidence, that Ethereum-valid execution can coexist with SRW3-invalid security state, and that the SRW3 decision can be made under a first-class, tamper-evident `SecurityContext` with replay-idempotent lineage. However, the SRW3 decision remained client-local/sidecar-level: nothing in any protocol semantics made an SRW3-invalid payload non-committable. Phase 1I attacks exactly that boundary:

What protocol semantics are required for an SRW3-invalid payload to become non-committable / non-canonical — without claiming an Ethereum consensus modification?

The central target is the implication

$$\text{Commit_P}(B, C, E, \text{Pi}) \Rightarrow \text{EthereumValid}(B) \text{ AND } \text{SRW3Valid}(B, C, E, \text{Pi})$$

and its contrapositive: an Ethereum-valid payload whose SRW3 evaluation rejects must be excluded from commitment. The phase does NOT assume the answer; it constructs the protocol model, mechanizes the theorem ladder in K, exercises the attack matrix, and demonstrates the required experiment over the real client boundary.

2. Starting assumptions (exposed, not hidden)

- A-G2 (protocol root authority) and A-G7 (policy governance authority) are carried over from Phase 1G unchanged. Phase 1I makes them EXPLICIT objects (the protocol configuration with a digest), but their authority is still protocol-given, not derived inside the model.
- The local SRW3 implementation (gate, evidence collection, policy loader)

is trusted code — the TCB disclosed since Phase 1H.

- The execution client is unmodified Geth 1.17.7 @ 3d858f85; its identity and configuration digest are client-derived and bound at the gate layer (SC-6/SC-7), not attested.
- K mechanization is at the level of the abstract protocol model; the keccak instantiation of digests is uninterpreted on the Haskell backend (finding 1C-1 discipline) and demonstrated on the Python mirror with real keccak (eth-utils).

3. Inherited results (Phase 1H-R1, verified before this phase)

The baseline was re-verified live before Phase 1I work began: LID lineage idempotence 11/11, SCT/AUTH/DET security-context 26/26, H1-H10 scenario suite 10/10 with exact (verdict, layer) match against the committed phase1h-r1-fix transcripts on a fresh unmodified Geth 1.17.7 devnet. The Level-III model consumes the Phase 1H-R1 gate verdicts and SecurityContext digest discipline unchanged (SRW3-CONTEXT-H-V1 continues as the context canonicalization domain).

4. The protocol model (Level III; spec section 4)

The model is implemented twice, deliberately: as executable Python (phase1i/python/pi_protocol.py, the exhaustive-experimentation oracle) and as a K definition (phase1i/semantics/srw3proto.k, the mechanized layer). Both implement exactly the same objects and predicates.

Objects: ProtocolConfig Pi (level-0 authority: chain id, fork version, the authorized policy commitment, the authorized execution-client identity, the authorized client configuration digest; ProtoRoot = H(0x9C || fields)), PolicyCommitment, SecurityContext (the 1H-R1 shape: policy version and digest, chain id, application-set digest, interaction-graph digest, lineage head, client execution configuration, client identity; contextDigest recomputed, never trusted), Evidence (semantic projection: digest, issuer, status, decision), ProtocolBlock B (parent commitment, payload digest, parent state root, post state root, effect digest, evidence, context, presented policy commitment, recorded context digest, slot), and ProtocolState S (committed head, committed chain, lineage head, height, active policy commitment, activation history, commitment records — records carry NO wall-clock field, the Phase 1H-R1 lesson carried upward).

Transition: $B_{\{n+1\}} = \text{Apply}(B_n, P_n)$ — deterministic; commit appends the commitment record, advances lineage to the committed payload identity, and

increments height. The transition is the **ONLY** rule that extends the chain (the $K_{pAccept}$ discipline), so "committed" has exactly one cause.

Validity: $Valid_P(B) = Valid_Exec(B) \text{ AND } Valid_SRW3(B)$. $Valid_Exec$ binds the block to the client-derived execution results (payload identity, parent/post state roots, effect digest). $Valid_SRW3$ requires an evidence object that is present, bound to the executed effect, issued by the protocol-authorized client, carrying a **VALID** decision, under a first-class-valid context (SC-1..SC-9 at the protocol layer, authorized values derived from the protocol config, never from the block; SC-10: no substitution path exists).

Commitment (the central predicate):

```
Commit_P(B) := Valid_Exec(B) AND Valid_SRW3(B)
AND ParentCommitted(B) AND ContextValid(B) AND PolicyValid(B)
AND EvidenceAvailable(B) AND SlotAvailable(B)
```

The spec's formula is explicitly non-exhaustive ("do not silently assume that these are the only conditions"); Phase 1I adds **ONE** documented additional conjunct: $SlotAvailable(B)$ — two conflicting blocks at the same protocol slot cannot both commit on the canonical chain (the one-block-per-slot rule; the Ethereum analog is proposer-slot uniqueness). The K transition model enforces the same rule structurally (the accept guard pins $pbSlotOf(B) == Int\ N$ and N advances only on accept). Every refusal names the failed conjunct.

5. Authority model (spec section 6)

The hierarchy Protocol/Consensus (0) > Policy (1) > Execution (2) > SRW3-evaluation (3) > Application (4) continues the frozen Phase 1G levels, with two non-negotiable properties made executable. **AUTHORITY DESCENT:** every authorization edge points strictly down; nothing authorizes itself. **CONTEXT-DERIVED AUTHORIZED IDENTITIES:** the authorized policy commitment, client identity, and client configuration are deterministic functions of the protocol configuration — never of the object being checked. The policy authority id is $H(0xA5 \ || \ chainId \ || \ forkVersion)$ — derived from the config, so a policy cannot carry its own authority (K claim PI5-AUTHORITY-DESCENT pins the construction). Negative controls (self-authorizing policy, self-authorizing evidence, policy substitution, policy-version substitution, authority-ID substitution, protocol-root substitution, security-context substitution) are mechanized in the attack matrix (PI-A5..A10) and the countermodel suite (CM-I1/I6/I7). The deliberately-tested **NEGATIVE** model ($PolicyAuthority = Application$) is

shown to break commitment safety: two application-chosen policies coexist and honest participants following different applications select different heads (GOV-5, CM-I1/CM-I5) — the necessity argument for the descent.

6. Policy commitment model (spec section 5)

``PolicyCommitment = H(0x9D || policyVersion || policyDigest || appSetDigest`

<code>graphDigest</code>	<code>policyAuthorityId)`</code> . The construction binds WHAT governs
--------------------------	--

(version + content digests), WHAT it governs (application surface), and WHO is authorized to govern (config-derived authority id). The `SecurityContext` is deliberately NOT a `PolicyCommitment` input: the context carries per-block mutable, client-derived state (lineage head, client config, client identity); folding it into the policy identity would (a) make policy governance change on every block, (b) let a substituted context masquerade as a policy change, and (c) put a client-derived object inside the protocol's policy root. The context is instead bound at commitment-validation time (`ContextValid(B)` via SC-1..SC-9 and the recorded context digest). The spec's literal form `H(..., SecurityContext)` is compared as option B of the commitment-root design (section 12 of this report); the selected construction is option A refined with the explicit authority id.

7. Commitment predicate and fork-choice semantics (spec section 9)

`Commit_P` is model-independent; the three fork-choice policies are enforcement surfaces of the SAME rule:

- Model A (reject): `SRW3 != VALID => protocol-invalid`; the payload is refused and never canonical. Strongest, matches the 1H consensus-visible simulation.
- Model B (non-canonical): `SRW3 != VALID => the payload stays Ethereum-valid but is ineligible for canonical head`. INSUFFICIENT ALONE (FC-6 result): head filtering grants no commitment authority — the guarantee lives inside `Commit_P`; a node that commits a non-canonical block violates nothing in Model B's own rules.
- Model C (deferred): `SRW3 UNKNOWN/ERROR => the block is a DEFERRED candidate that cannot finalize until a decision exists`; `SRW3 REJECT => security-invalid (non-committable)`. The liveness-friendliest failure mode: it names the deferral explicitly instead of silently refusing.

Comparison results (mechanized in `run_forkchoice_compare.py`, FC-1..FC-7): safety is carried by `Commit_P` in all models; honest proposals commit in

all models; under an evidence-infrastructure outage A/B refuse and C defers — progress stops in ALL models (CM-I8), which is the honest cost of fail-closed mandatory SRW3, not of any one model; determinism holds in all models; failure behavior is distinguishable per model. The MINIMAL protocol rule identified: `Commit_P` as defined in section 4, fail-closed on evidence availability, with the fork-choice surface chosen per deployment (A for strictness, C for liveness transparency). Model B is documented as necessary-but-not-sufficient (it is the natural Ethereum fork-choice integration surface, but only because `Commit_P` carries the guarantee).

8. Evidence-availability semantics (spec section 10)

The chosen protocol rule is FAIL-CLOSED, with exactly one refinement: a missing/unavailable/malformed/inconsistent evidence object NEVER yields `Valid_SRW3` (by construction — the predicate requires status "present" and a VALID decision), and when the ONLY failures are the security-evaluation ones with no decision in existence, the predicate returns DEFERRED (the "cannot commit YET" state) rather than REFUSED (an active negative). The terminal treatment is the fork-choice model's choice (A: refused; B: non-canonical; C: deferred). `EvidenceMissing(B) => NOT SRW3Valid(B)` holds by construction and is tested (PI-A11 across all three models). This is the mechanization of "a missing security proof object must not silently become a valid security result."

9. Formal theorem ladder (spec sections 7/8/19)

Ladder: PI-0 definitions (`srw3proto.k`) > PI-1 validity composition > PI-2 commitment exclusion > PI-3 deterministic commitment (definitional; digest layer demonstrated) > PI-4 lineage continuity (inherited PROVED from Phase 1D) > PI-5 policy authority > PI-6 reorg/replay safety (transition level + Python) > PI-7 committed-state safety.

K/KEVM availability: K v7.1.337 was RE-PROVISIONED in this environment from the pinned release (`jammy .deb`, extracted without root) with z3 4.13.3 (official binary) and flex 2.6.4. `srw3proto.k` kompiles cleanly on the Haskell backend against the frozen 1G chain (`srw3authz.k` required unchanged; includes 1F/1E/1D semantics via the pinned `-I` paths).

Anti-vacuity: the accept premise of PI-1 is witnessed concretely by the required experiment (a real Geth payload evaluating to `Commit_P = COMMIT`), and the gate accept ("valid-g", 21 atomic conjuncts) was witnessed in Phase 1G.

PROVED BY K (Haskell backend, kprove; 12/12 mechanizable claims):

Claim	Content
PI1-COMMIT-SAFETY	accept direction: all four conjuncts => Commit_P true (no acceptance slack)
PI2-DECISION-REJECT	gate-derived REJECT decision => Commit_P false
PI2-DECISION-UNKNOWN	missing/deferred decision => Commit_P false (never silently valid)
PI2-EXEC-INVALID	execution-invalid => Commit_P false
PIB-GATELINK-VALID	only the frozen 21-conjunct "valid-g" maps to the valid protocol decision
PIB-GATELINK-REJECT1..3	authority-type / authority-source / policy rejection verdicts exclude commitment (frozen-semantics linkage)
PI5-AUTHORITY-DESCENDANT	the authorized commitment is a function of the level-0 config alone (no block-supplied input)
PI7-BASE	the genesis chain is chain-valid
PI7-STEP	one commitment-bound accept preserves chain validity (parent link, next slot, Commit_P)
PI7-REJECT	a block failing the security verdict (or the structural link) cannot extend the chain — state unchanged

NOT MECHANIZED (disclosed with exact reasons):

- PI2-POLICY-SUB / PI2-CTX-SUB (policy-commitment / context-digest substitution exclusion): the requires

`notBool(X ==K PolicyCommitmentI(C))` cannot be simplified because `Keccak256raw` has NO evaluators on the Haskell backend (finding 1C-1) and the prover aborts while simplifying the claim. Both exclusions are DEMONSTRATED exhaustively at the model level (Python attack matrix PI-A5/A6 — including the re-recorded-substitution variant — and PI-A7), and the positive direction is pinned by PI1-COMMIT-SAFETY.

- PI7-ALL (forall-n circularity closure): the base, step and reject claims are PROVED; the `[circularity]` closure aborts with the kore message "The configuration's term unifies with the destination's term, but the implication check between the conditions has failed" (the accept rule's side conditions enter the hypothesis check). The closure content (every reachable chain is commitment-bound) is DEMONSTRATED by the exhaustive Python suites. Attempt retained (`proofs/pi7all_only.k`).

- LLVM digest-instantiation demo: the LLVM-15 backend toolchain was NOT re-provisioned in this environment; the keccak-instantiated digest constructions are demonstrated by the Python mirror with real keccak (DET-K1..K3), computed over exactly the byte layouts defined in `srw3proto.k` (tags 0x9C/0x9D/0x9E/0xA5/0xA6).

No "proved" claim depends on Python tests; no Python suite is claimed as proof.

10. Attack matrix (spec section 17)

PI-A1..A16 mechanized over the Level-III model — 30/30 assertions PASS (including sub-cases and the fork-choice cells): the four Ethereum x SRW3 validity cells (A1 COMMIT; A2/A3/A4 refused with the failed conjunct named); policy substitution (A5); policy-digest substitution including the re-recorded variant (A6: SC-9 catches the post-hoc tamper, SC-2 catches the re-record); stale context (A7, SC-8); stale parent root (A8); evidence substitution (A9, effect binding); self-authorizing evidence (A10, authsrc); missing evidence per the explicitly chosen rule (A11: fail-closed, Model A refused / B non-canonical / C deferred); reordered encoding (A12: identical decision); replayed payload (A13: identical commitment semantics, exactly one record); reorg + stale evidence (A14: refused, displaced record byte-stable, honest continuation commits); two clients with equivalent execution (A15: identical SRW3 result and commitment); three-way interaction violation (A16: refused).

11. Countermodels (spec section 18) — retained results

CM-I1 (policy authority ambiguity): two valid-looking authorities produce different commitments; the protocol pin decides — the application-chosen authority is REFUTED as sufficient. CM-I2 (evidence availability): one participant commits while another cannot evaluate; the unequipped participant never falsely accepts (fail-closed), at the cost of a liveness/split risk that Model C names explicitly as DEFERRED. CM-I3 (client divergence): equivalent execution => equivalent evidence => same decision (PI-A15); a divergent post-state is Ethereum-invalid BEFORE SRW3 runs (post-state binding). CM-I4 (policy upgrade retroactivity): committed blocks remain committed — the stronger conjecture "current policy re-validates committed history" is REFUTED as a design requirement (history immutability is the chosen semantics). CM-I5 (fork-choice split): different policy versions across honest participants yield different heads; prevented ONLY by the protocol-distributed single policy commitment (a stale-config participant is by definition not following the protocol — a governance-failure mode, disclosed). CM-I6 (governance capture): an application modifying the invariant set while appearing protocol-authorized is caught (SC-4/SC-9); under the deliberately-tested negative model the capture SUCCEEDS — the necessity argument. CM-I7 (security-root circularity): ``SRW3Root = H(0x9F || parentRoot || policyCommitment || contextDigest || evidenceDigest || decision)`` is over FIXED fields — self-reference is unconstructible; the root's authority derives from the protocol config pin, never from the root itself. CM-I8 (liveness failure):

mandatory fail-closed SRW3 stalls commitment when the evidence infrastructure fails, under ALL three models — an honest, disclosed deployment cost.

12. Commitment-root design (spec section 16) — comparison, no auto-D

Candidate A (policy commitment only): minimal bandwidth, anchors WHAT governs, does not bind per-block evidence — substitution of evidence is invisible to the root itself. Candidate B (security-context commitment): binds the evaluation context per block; context digests already exist (1H-R1). Candidate C (security-evidence commitment): binds WHAT was decided and on WHICH evidence. Candidate D (combined `SRW3Root = H(parentRoot, policyCommitment, contextDigest, evidenceDigest, decision)`): defined and implemented for comparison (srw3_root, tag 0x9F), NOT auto-selected. The Level-III commitment record ALREADY binds all D inputs (block commitment canon 0x9E), so D adds no authority by itself — the authority comes from the protocol config pinning the policy commitment and the authorized client, not from embedding more digests. Selected: the block-commitment record (0x9E) as the commitment identity, with the policy commitment anchored at the protocol config (A's role), the context digest bound per block (B's role) and the evidence digest bound per block (C's role). A header-embedded SRW3Root (D) is the deployment path IF a consensus-level integration is ever attempted — out of scope here.

13. Deterministic encoding (spec section 11)

Canonical form: domain tag + sorted k=v lines over string-normalized fields (ints in decimal, bytes as 0x-hex) — the 1H-R1 context-digest discipline extended to protocol objects. Mechanized (DET-I1..I9, 15/15): field-order invariance; no wall-clock/process/order dependence; integer re-encoding normalized before digesting; JSON field-order invariance; duplicate-key raw serialization detected as MALFORMED (never silently canonicalized); missing optional fields keep the semantic digest; policy version mismatch changes the commitment; context-digest mismatch detected (SC-9 analog); two independent honest participants derive the same decision, validity, commitment and post-commit state.

14. Reorg and replay (spec sections 12/13)

Mechanized (RR-1..6, RP-1..4): competing branches both commit on their own

branch; reorg moves the head while displaced records stay byte-stable;
 stale evidence from a displaced branch is refused on the new branch
 (parentCommitment binding — cached decisions are NOT reused across reorg);
 the lineage head follows the new canonical tip; honest continuation
 commits. Replay: Evaluate(B, C, E, Pi) is a pure function of its inputs —
 idempotent under repetition, wall-clock absence, and construction order;
 re-recording is idempotent (exactly one record — the 1H-R1
 semantic_record discipline carried to the protocol layer); replay of a
 DISPLACED block post-reorg is refused (slot already committed on the
 canonical chain — the SlotAvailable conjunct).

15. Multi-client semantic requirement (spec section 14)

The abstract client interface (ClientProtocol) with two deterministic
 implementations states the requirement BEFORE any real multi-client work:

Client_A.execute(B) == Client_B.execute(B) (post-state), evidence equal
 under EquivalentEvidence (equality of the semantic projection: payload
 identity, parent/post state roots, effect digest — modulo client-local,
 security-irrelevant presentation), therefore the same SRW3 decision and the
 same commitment (PI-A15, CM-I3). Equivalent evidence is DEFINED, and the
 theorem-tested consequence holds: equivalent evidence produces the same
 SRW3 validity decision. Divergent execution (different post-state) is
 rejected by the Ethereum layer's own state-root binding before SRW3 runs.

16. Real-client prototype and the required experiment (spec sections 21/22)

The prototype is a protocol-level commitment SIMULATION over a real
 execution-client boundary: Geth 1.17.7-stable @
 3d858f858a458effb2a563788aedf1fe65e1f0d3 (unmodified; re-provisioned from
 the pinned gethstore tarball and verified) runs in shadow mode; the real
 harness collects client-derived evidence through the Engine API; the real
 gate evaluates it; and the Level-III Commit_P — consuming the REAL
 verdicts, evidence digests, context digests and state roots — decides
 commitment. It is NOT "Ethereum consensus enforcement", NOT a client
 modification, and NOT a protocol change.

Required experiment result — PASS:

Block	Ethereum	SRW3	Protocol commitment
EXP-BASE (price + deposit)	VALID	SRW3_VALID	COMMIT (A/B/C)

Block	Ethereum	SRW3	Protocol commitment
EXP-B' (single borrow 10 ETH, cap respected)	VALID	SRW3_VALID	COMMIT (A/B/C)
EXP-B (single borrow 500 ETH, cap violated)	VALID	SRW3_REJECT	REFUSED (A/C), NON-CANONICAL (B)

B and B' are the SAME transaction kind (a single borrow against the same deployed stack, both Ethereum-valid); they differ ONLY in the security-relevant condition (the aggregate-cap invariant respected vs violated).

The protocol model — not the sidecar — determines commitment. Shadow-mode ordering note: because the client executes every payload, the respecting borrow must run before the violating one (the violating borrow pushes `totalBorrowed` over the cap and would otherwise contaminate the next block's evaluation) — disclosed in the transcript.

17. Performance (spec section 23)

Measured SEPARATELY per phase (5 repetitions per class; median over 5; fresh unmodified Geth devnet; containerized sandbox, 2 vCPU; payload classes small=1 tx / medium=5 / large=20 price writes):

Class	execution_ms (client residual)	evidence_ms	srw3_ms	commitment_ms	total_ms
small	1675.6	7.7	0.11	0.15	1683.2
medium	1881.9	6.4	0.11	0.18	1894.4
large	2700.4	11.1	0.12	0.12	2710.3

`execution_ms` is the client-execution residual of the end-to-end pipeline (real Engine API payload production minus the separately measured evidence/srw3/commitment phases) and is dominated by Geth's payload build + Engine-API round trips (the Phase 1H envelope). The SRW3 gate (including SC validation) adds ~0.11-0.12 ms and the protocol commitment predicate ~0.12-0.18 ms — both are 3-4 orders of magnitude below execution. No single aggregate "Ethereum overhead" percentage is reported, per spec.

18. Governance / upgrade analysis (spec section 24)

Mechanized (GOV-1..5): a protocol-authoritative upgrade changes the config's authorized policy commitment and appends to the activation history; new blocks under the upgraded policy commit; blocks still presenting the superseded commitment refuse; committed history is IMMUTABLE under upgrade (CM-I4's answer: no retroactive invalidation); rollback is impossible — the activation history is append-only and

re-presenting the superseded commitment (rollback-by-presentation) refuses even with a fully rebuilt context; an application cannot bypass policy (invariant-set changes are commitment-changing and caught); the negative model (PolicyAuthority = Application) is deliberately exercised and shown to produce divergent commitment heads — the reason the policy commitment MUST be pinned at the level-0 protocol configuration. Policy activation needing lineage: the activation history is part of the protocol state and moves with the committed chain.

19. Security-root versus state-root (spec section 25)

Three distinct roots with distinct guarantees: StateRoot attests the execution state (Ethereum's own commitment — untouched); PolicyRoot (the protocol config root, 0x9C) attests WHICH policy and client are authorized — the level-0 authority anchor; the SRW3 commitment record (0x9E) / candidate SRW3Root (0x9F) attest the SECURITY validity of the transition (decision + context + evidence under the pinned policy). The SRW3 root does NOT replace the state root; it is an ADDITIONAL commitment layer: Block -> StateRoot AND SRW3Commitment. What each actually provides: the state root proves state consistency of execution; the policy root proves authorization descent; the SRW3 root proves that the committed transition satisfied the security policy — neither subsumes another.

20. Ethereum compatibility analysis (spec section 15)

Existing Ethereum behavior: payload validity is execution-layer validity (state-root binding, gas, etc.); the consensus layer commits the head the fork-choice selects; NOTHING in current Ethereum evaluates an application-level security policy at commitment time (demonstrated throughout Phases 1E-1I: Ethereum-valid + SRW3-invalid is reachable and was committed in shadow mode before this phase's protocol layer is applied). Proposed SRW3 protocol rule: Commit_P as defined — a NEW protocol validity predicate. Research prototype: the Level-III simulator + the real-client commitment simulation of section 16. Candidate intervention points, assessed: Engine-API modification alone does NOT create consensus authority (the CL could ignore an adapter's answer); fork-choice eligibility filtering is Model B — necessary-but-not-sufficient; the MINIMUM intervention point capable of enforcing ExecValid AND NOT SRW3Valid => NOT Commit is the protocol's commitment/validity predicate itself (Model A semantics at Level III), with fork-choice filtering (B) and deferred finality (C) as enforcement

surfaces of the same rule. An actual Ethereum integration would additionally require a new protocol commitment rule (header commitment / fork-choice parameterization) — Level IV, explicitly OUT of scope and NOT claimed.

21. Limitations

The protocol model is a Level-III abstract model: it does not change Ethereum, does not modify Geth, and does not create consensus-visible enforcement. The K mechanization covers the abstract layer; keccak is uninterpreted there and demonstrated on the Python mirror; the forall-n circularity closure (PI7-ALL) is not machine-closed in this environment. Policy authority is protocol-given (A-G7) — WHO governs the governance remains outside the model. Evidence availability is assumed to be ultimately reachable (Model C's deferral must resolve eventually) — the liveness cost of fail-closed operation is disclosed, not solved. The multi-client requirement is specified and tested against deterministic client ABSTRACTIONS, not against two real execution clients. Performance was measured on a containerized sandbox, not production hardware.

22. Exact scientific claims

PROVED (K, Haskell backend): the 12 claims of section 9 — the commitment predicate's accept direction, the decision-level and execution-level exclusions, the frozen-gate linkage, the policy-authority descent, and the committed-chain base/step/reject safety — within the abstract protocol model, with crypto hooks uninterpreted.

DEMONSTRATED (experimentally, over the executable model and/or the real client boundary): the central implication's contrapositive in the Ethereum-valid + SRW3-invalid cell (required experiment); the full attack matrix PI-A1..A16; countermodels CM-I1..I8; determinism DET-I1..I9 + K-mirror; reorg/replay RR-1..6 + RP-1..4; governance GOV-1..5; fork-choice comparison FC-1..7; the keccak-instantiated digest constructions (DET-K1..K3); zero regression against Phase 1H-R1.

ASSUMED: A-G2/A-G7 (protocol root and policy governance authority, now explicit config objects); the local SRW3 implementation TCB; unmodified client identity binding (no attestation).

REFUTED: Model B as a SUFFICIENT enforcement surface (FC-6); retroactive policy re-validation of committed history as a requirement (CM-I4); PolicyAuthority = Application (GOV-5/CM-I1/I5); "context digest inside the policy commitment" as the policy identity construction (section 6

justification; compared as root option B).

NOT MECHANIZED: PI2-POLICY-SUB / PI2-CTX-SUB in K (keccak evaluator absence, exact abort disclosed); PI7-ALL forall-closure (kore circularity implication check, exact message disclosed); the LLVM digest-instantiation demo (toolchain not re-provisioned).

23. Stopping condition and next unresolved boundary

Outcome B — QUALIFIED POSITIVE. The protocol-level commitment rule is formally specified AND mechanized in the bounded model (12/12 mechanizable K claims; the central implication's exclusion direction proved at the decision, gate-linkage and execution levels; the policy/context-digest substitution branches and the forall-closure disclosed as not mechanized and demonstrated instead), it is deterministic, authority-safe and reorg/replay-safe, and it was demonstrated end-to-end over a real, unmodified execution-client boundary. Deployment assumptions remain: protocol-authoritative policy distribution (WHO runs the governance), an evidence-availability infrastructure with bounded deferral, and the actual Ethereum protocol/fork integration (Level IV) that would carry the commitment predicate into consensus — explicitly out of scope and not claimed. Per the stopping condition, Phase 1I STOPS here; no further phase is started.